Final Review Project Report

on

"***Custom API gateway with JWT & rate limiting***"

submitted by

Satyam Gupta

Email ID – bt23eci034@iiitn.ac.in

from

Department of Electronics & Communication

Indian Institute of Information Technology, Nagpur (IIIT Nagpur)

भारतीय सूचना प्रौद्योगिकी संस्थान, नागपुर

Indian Institute of Information Technology, Nagpur

(*An Institution of National Importance by an Act of Parliament*)

Nagpur– 441108, India

DECLARATION

I, Mr. Satyam Gupta, hereby declare that the project report entitled "**Custom API gateway with JWT & rate limiting**" is an original work carried out and prepared by me.

I affirm that this project and its accompanying documentation are the results of my own efforts, research, and implementation. Any external references, existing works, or code snippets used for guidance have been properly cited and acknowledged in this report.

I fully understand the significance of this declaration and hereby certify that all information, results, and analyses presented in this report are true and accurate to the best of my knowledge and belief.

Name - Satyam Gupta

Email ID - bt23eci034@iiitn.ac.in

ACKNOWLEDGEMENT

The successful completion of my project, **"Custom API gateway with JWT & rate limiting"**, has been a highly enriching and educational experience. I am deeply grateful to the **Indian Institute of Information Technology, Nagpur** for providing me with the opportunity, infrastructure, and guidance to carry out this work as part of my academic learning and technical development.

I would like to express my heartfelt gratitude to the **Zoho Team** for their valuable insights and practical exposure to real-world software development practices, which greatly enhanced my understanding of microservices, deployment strategies, and observability tools. Their guidance played a crucial role in shaping this project into a functional and well-structured implementation.

I wish to extend my sincere thanks to my **faculty members and mentors at IIIT Nagpur** for their continuous support, feedback, and encouragement throughout the duration of this project. Their expertise and constructive suggestions were instrumental in refining both the technical and analytical aspects of my work.

Finally, I would like to thank my **friends and peers** for their motivation, discussions, and teamwork during this journey. Their collaboration and assistance made this project both productive and enjoyable.

This project marks an important milestone in my learning journey, and I am committed to applying the knowledge and skills gained through this experience in my future academic and professional endeavors.

Satyam Gupta
Indian Institute of Information Technology, Nagpur

ABSTRACT

This project presents the design and implementation of a **Custom API Gateway** that acts as a central entry point for microservices, ensuring secure and efficient communication. The system integrates **JWT and API Key authentication**, **rate limiting**, **logging**, and a **round-robin load balancer** for traffic distribution. It connects with **MongoDB** for data management and uses **Grafana** and **InfluxDB** for monitoring and visualization. Developed using **Node.js and Express.js**, the project demonstrates scalability, reliability, and observability in a microservice environment, making it a lightweight alternative to existing gateway solutions.

LEARNING OBJECTIVE OF PROJECT

The primary learning objectives of this project, **"Custom API gateway with JWT & rate limiting"** are as follows:

1. **To understand the architecture of microservice-based systems** and the role of API Gateways in managing communication between clients and services.
2. **To learn and implement authentication mechanisms** such as **JWT (JSON Web Token)** and **API Key-based security** for secure access control.
3. **To develop and integrate a custom load balancing algorithm** (Round Robin) to efficiently distribute network traffic among multiple service instances.
4. **To implement rate limiting and logging mechanisms** for improved performance, reliability, and security of API requests.
5. **To gain hands-on experience in backend development** using **Node.js, Express.js, and MongoDB** for scalable and modular system design.
6. **To explore observability tools** such as **Grafana, InfluxDB, and K6** for real-time monitoring, data visualization, and performance testing.
7. **To enhance deployment and scalability skills** by setting up the microservices and API Gateway for container-based or Kubernetes environments.
8. **To apply DevOps principles** for integrating monitoring, automation, and fault tolerance into microservice ecosystems.

INDEX

S.no.	Particular	Page No.
1.	Abstract	4
2.	Introduction	7
3.	Methodology	8-17
4.	Setup and Execution	18-22
5.	Results	23-31
6.	Conclusions	32
7.	Future Enhancements	33
8.	References	34

INTRODUCTION

This project, titled **"Custom API gateway with JWT & rate limiting"** focuses on designing and implementing a secure and efficient API Gateway that serves as a single entry point for multiple microservices. The goal is to create a lightweight and extensible system that ensures smooth communication, authentication, and monitoring across distributed services.

With the growing adoption of **microservice architecture**, applications are divided into smaller, independent services that communicate through APIs. Managing these services securely and efficiently requires a gateway that can handle authentication, authorization, rate limiting, and load balancing. This project focuses on creating a **custom-built gateway** instead of relying on heavy third-party tools, making it simpler and easier to extend.

The **API Gateway** was implemented using **Node.js and Express.js**, and it connects with **MongoDB** to manage users, API keys, and request logs. It supports both **JWT** and **API Key-based authentication**, ensuring secure access to microservices. A **round-robin load balancer** was implemented to distribute requests evenly across multiple instances of the payment and refund services.

For monitoring and performance analysis, the project integrates **Grafana** and **InfluxDB**, which visualize real-time metrics collected through **K6 load testing**. This setup allows for evaluating system performance under different traffic loads, ensuring reliability and scalability.

Overall, this project helped me gain practical experience in backend development, API management, and observability practices. It provided a clear understanding of how large-scale applications manage communication, authentication, and traffic distribution efficiently using modern software architecture concepts.

METHODOLOGY

The methodology adopted for this project focuses on the design, development, and implementation of a **Custom API Gateway** that manages secure and efficient communication among multiple microservices. The project includes steps such as system architecture design, backend development, database integration, observability setup, and testing.

The system is built using **Node.js** and **Express.js** and deployed in a simulated microservice environment with **MongoDB**, **Grafana**, **InfluxDB**, and **K6** for performance testing and monitoring.

2.1 System Architecture

The architecture follows the **Microservice Communication Model**, where the **API Gateway** acts as the single entry point for all client requests. It is responsible for authentication, rate limiting, load balancing, and request logging.

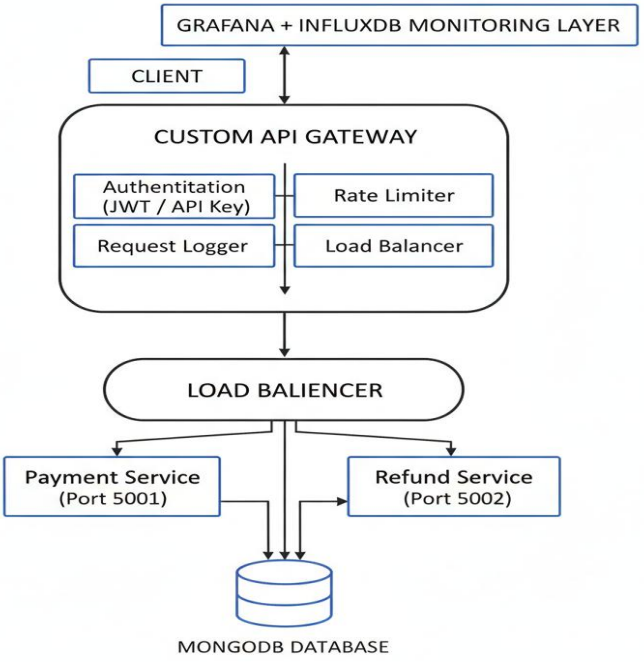


Figure 1 : System Architecture

The gateway processes requests, applies authentication and rate limits, forwards requests to the respective microservice, stores logs in the database, and monitors system performance using integrated tools.

Workflow

1. The client sends a request to the API Gateway using either a Bearer Token (JWT) or an API Key.
2. The authentication module validates the credentials using the auth.js middleware.
3. The rate limiter (rateLimiter.js) ensures users do not exceed a set number of requests per time window.
4. Once validated, the gateway forwards the request to either the Payment Service or the Refund Service using Axios.
5. The load balancer (loadBalancer.js) distributes requests evenly across multiple service instances using the Round Robin algorithm.
6. Request and response data, including IP, path, method, and status, are stored in MongoDB via the RequestLog model.
7. Performance metrics are collected using K6, stored in InfluxDB, and visualized in Grafana dashboards.

2.2 Folder Structure

```
Custom-API-Gateway/
│
├── middleware/
│   ├── auth.js            → Handles JWT and API Key authentication
│   └── rateLimiter.js     → Implements express-rate-limit based throttling
│
├── models/
│   ├── ApiKey.js          → Schema for API keys linked with users
│   ├── User.js            → User schema with hashed passwords
│   ├── Payment.js         → Stores payment transactions
│   ├── Refund.js          → Handles refund records
│   └── RequestLog.js      → Logs incoming API requests
│
├── services/
│   ├── paymentService.js  → Handles payment creation and status
│   └── refundService.js   → Handles refunds and transaction listings
│
├── scripts/
│   ├── seed.js            → Seeds MongoDB with a test user and API key
```

```
│   └── generateToken.js     → Generates a JWT for testing authentication
│
├── utils/
│   ├── db.js                → MongoDB connection handler
│   ├── loadBalancer.js      → Round Robin load balancing logic
│   └── logger.js            → Configured using Winston for centralized logging
│
├── tests/
│   └── load_test.js         → K6 script for load/stress testing
│
├── apiGateway.js            → Main Express app (gateway entry point)
├── docker-compose.yml       → Docker setup for Grafana, InfluxDB, and K6
├── .env                     → Environment variables (JWT_SECRET, MONGO_URI)
└── package.json             → Project dependencies and scripts
```

2.3 Setup and Configuration

The project setup began by installing Node.js, Express.js, and MongoDB. Environment variables such as MONGO_URI and JWT_SECRET were configured in a .env file.

The Winston logger was added for structured logging, and Mongoose schemas were created for managing users, API keys, payments, refunds, and request logs.

The setup also included integrating morgan for HTTP request logging and establishing database connections via db.js

.2.4 Authentication

Two authentication mechanisms were implemented — **JWT (JSON Web Token)** and **API Key-based authentication**.

- **JWT Authentication** verifies the user's identity using a signed token. If the token is valid and unexpired, the user is granted access to protected endpoints.
- **API Key Authentication** checks for an active key stored in MongoDB, updates the last_used_at field, and allows the associated user to access services.

This modular approach allows both end-users and services to authenticate securely with the API Gateway.

2.5 Rate Limiting

To protect the system from abuse and excessive requests, **rate limiting** was implemented using the express-rate-limit library.

Each client IP is restricted to **100 requests** per hour, and violations are logged in the system with detailed messages.

The /health endpoint is excluded to ensure continuous system monitoring and testing access.

2.6 Load Balancing and Routing

A custom **Round Robin Load Balancer** was developed in loadBalancer.js to distribute requests evenly between available microservice instances.

This ensures equal load distribution and prevents any single service from becoming overloaded.

Routing logic was implemented using Axios inside the API Gateway, dynamically forwarding requests to the correct service endpoint based on the route (/api/payments or /api/refunds).

2.7 Microservice Implementation

To demonstrate the routing, authentication, and load balancing capabilities of the **Custom API Gateway**, two independent microservices were developed — the **Payment Service** and the **Refund Service**. Each service was designed to function as a self-contained application with its own logic, endpoints, and database interactions. These services are hosted on separate ports and communicate with the gateway through RESTful APIs.

1) Payment Service (Port 5001)

The **Payment Service** is responsible for handling all payment-related operations. It provides endpoints for initiating new payments and for checking the status of existing transactions. The service operates independently but connects to the shared MongoDB database to store and retrieve transaction data.

Key Functionalities:

1. **/pay Endpoint:**
    - Accepts a POST request containing details such as userId, amount, currency, and paymentMethod.
    - Validates all required fields before processing the transaction.
    - Once validated, it creates a new payment record in the **Payment** collection in MongoDB using the Payment.js model.
    - The transaction is marked as "completed" after successful insertion, and a response is returned containing the payment ID, amount, currency, and status.
2. **/status/:id Endpoint:**
    - Accepts a GET request with the payment ID as a URL parameter.

- Retrieves the payment details from the database and returns the transaction's current status and other metadata.
- If the payment ID does not exist, the service responds with an appropriate error message.

3. **/health Endpoint:**
   - Returns a simple JSON response confirming that the service is operational (e.g., { "status": "healthy", "service": "payment-service-1", "port": 5001 }).
   - This endpoint is primarily used by the gateway and monitoring tools to perform **health checks**.

Data Management:

Each payment record is stored in MongoDB with fields such as:

- user_id (references the User model)
- amount
- currency
- status
- payment_method
- service_instance
- timestamps

2) Refund Service (Port 5002)

The **Refund Service** handles all refund-related operations and interacts with the Payment Service through shared references in the database. Its primary role is to process refund requests and provide a list of user transactions when needed.

Key Functionalities:

1. **/refund Endpoint:**
    - Accepts a POST request with parameters such as paymentId, amount, and reason.
    - Verifies that the provided payment ID exists in the Payment collection.
    - Ensures that the refund amount does not exceed the original payment amount.
    - Creates a new refund record in the **Refund** collection using the Refund.js model, marking it as “completed” once successfully processed.
    - Responds with refund details including refund ID, payment ID, amount, and status.
2. **/transactions Endpoint:**
    - Accepts a GET request and supports query parameters like userId, limit, and offset.
    - Retrieves all transactions associated with the provided user ID from the Payment collection.
    - Returns a paginated list of transactions to improve performance and manage large datasets efficiently.
3. **/health Endpoint:**
    - Confirms the operational status of the service with a simple response such as { "status": "healthy", "service": "refund-service-2", "port": 5002 }.
    - Used for readiness and liveness checks by the API Gateway and monitoring stack.

2.8 Logging and Monitoring

Every incoming request and response is logged using the RequestLog model. Details such as method, path, response time, IP address, and status code are recorded.

Logs are displayed in real-time through the Winston Logger and stored persistently in MongoDB for auditing and analysis.

Monitoring and visualization were achieved using:

- **Grafana:** For real-time dashboards.
- **InfluxDB:** For time-series metrics storage.
- **K6:** For generating load and stress test data.

The **Docker Compose setup** creates containers for Grafana, InfluxDB, and K6, enabling automated monitoring and analysis.

2.9 Testing and Validation

Comprehensive testing was conducted to ensure the system works as expected:

- **Functional Testing:** Verified all routes, authentication checks, and rate limits.
- **Performance Testing:** Used **K6** to simulate concurrent requests and measure system performance.
- **Monitoring Validation:** Observed metrics in Grafana to ensure stable throughput, low error rates, and quick response times under varying traffic.

The system demonstrated consistent performance with low latency and effective traffic distribution.

2.10 Tools and Technologies Used

1. Node.js – Used as the main runtime environment for building the API Gateway and microservices.
2. Express.js – Web framework used for creating REST APIs and handling routing efficiently.
3. MongoDB – NoSQL database used to store user data, API keys, payments, refunds, and request

logs.

4. Mongoose – ODM (Object Data Modeling) library used to define schemas and interact with MongoDB.
5. JWT (JSON Web Token) – Used for secure authentication and authorization of API users.
6. API Key Authentication – Provides an alternative method for authenticating service-to-service requests.
7. Express-Rate-Limit – Used to implement rate limiting and control the number of requests per user or IP.
8. Axios – HTTP client used by the gateway to forward and handle API requests between microservices.
9. Winston – Logging library used to record system events, errors, and activity logs.
10. Morgan – Middleware used for HTTP request logging in the gateway.
11. Grafana – Visualization tool used for real-time monitoring of performance metrics.
12. InfluxDB – Time-series database used to store and analyze performance data collected during testing.
13. K6 – Load testing tool used to measure throughput, response time, and stability of the system.
14. Docker Compose – Used to containerize and run Grafana, InfluxDB, and K6 together for observability setup.
15. dotenv – Used to manage environment variables securely within the project.
16. bcrypt.js – Used for hashing user passwords during database seeding and authentication.

Setup and Execution

To demonstrate the routing, authentication, and load balancing capabilities of the **Custom API Gateway**, two independent microservices were developed — the **Payment Service** and the **Refund Service**. Each service was designed to function as a self-contained application with its own logic, endpoints, and database interactions. These services are hosted on separate ports and communicate with the gateway through RESTful APIs.

3.1 System Overview

The project is composed of three main components that interact with each other through HTTP-based communication:

- **API Gateway** — Runs on port **4000** and acts as the central entry point for all client requests. It handles authentication, routing, rate limiting, and logging.
- **Payment Service 1** — Runs on port **5001** and handles all payment-related operations.
- **Refund Service 2** — Runs on port **5002** and manages refund transactions and transaction queries.

All three components operate independently but communicate through RESTful APIs. The API Gateway forwards requests to the respective services after validating authentication and enforcing rate limits.

3.2 Available Endpoints

Payment Service 1 (Port 5001)

- POST /pay – Creates and processes a new payment transaction.
- GET /status/:id – Retrieves the status of a specific payment using its ID.
- GET /health – Returns a service health check confirming that the service is active and

accessible.

Refund Service 2 (Port 5002)

- POST /refund – Processes refund requests for existing payments.
- GET /transactions – Returns a list of transactions for a particular user.
- GET /health – Verifies the operational status of the refund service.

API Gateway (Port 4000)

- POST /api/payments/pay – Forwards payment requests to **Payment Service 1**.
- GET /api/payments/status/:id – Retrieves payment status from **Payment Service 1**.
- POST /api/refunds/refund – Routes refund requests to **Refund Service 2**.
- GET /api/refunds/transactions – Fetches user transaction data through **Refund Service 2**.
- GET /health – Checks the API Gateway's operational and service status.

The **/health** endpoints across all services are particularly useful for deployment validation, load balancer readiness checks, and monitoring through Grafana or Kubernetes.

3.3 Installation Steps

Step 1: Install Project Dependencies

Open the terminal in the project root directory and run the following command:

➔ npm install

This command installs all required dependencies for the API Gateway and the microservices, including Express, Mongoose, Winston, Axios, and supporting middleware libraries.

Step 2: Seed the Database

Before starting the system, seed the **MongoDB** database with a default test user and API key:

➔ npm run seed

This command executes the scripts/seed.js file, which automatically:

- Creates a test user with an email and hashed password in the **User** collection.
- Generates a new API key stored in the **ApiKey** collection.

This data will be used for authentication testing.

Step 3: Generate a JWT Token

After seeding the database, generate a **JWT Token** for the test user by running:

➔ npm run generate-token

This command executes scripts/generateToken.js and prints a valid **JWT token** to the terminal. The generated token will be used for making authenticated API requests via the gateway.

Step 4: Configure Environment Variables

Create a .env file in the root directory of the project and add the following configuration variables:

- MONGO_URI=mongodb+srv://satyam_db_use:Satyam123!@cluster0.kiywgf4.mongodb.net/
- PORT=4000
- JWT_SECRET=supersecretkey123
- DASHBOARD_PORT=7000

This configuration ensures a secure and reliable connection to the database and proper authentication handling within the gateway.

3.4 Running the System

Starting All Services Together

To start all components simultaneously (API Gateway, Payment Service 1, and Refund Service 2), run:

➔ npm start

This command launches all services using **start.js** with predefined ports:

- API Gateway → **Port 4000**
- Payment Service 1 → **Port 5001**
- Refund Service 2 → **Port 5002**

Once running, the gateway and microservices begin handling requests and communicating internally.

3.5 Authentication Methods

The API Gateway supports **two authentication mechanisms** for accessing secured endpoints.

1. JWT Token (Bearer Authentication)

Requests can be made using a JWT token in the Authorization header:

```
curl -X POST http://localhost:4000/api/payments/pay \
  -H "Authorization: Bearer YOUR_JWT_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "userId": "USER_ID",
```

```
  "amount": 99.99,
  "currency": "USD",
  "paymentMethod": "credit_card"
}'
```

2. API Key Authentication

Alternatively, use the API key for service-level authentication:

```
curl -X POST http://localhost:4000/api/payments/pay \
  -H "X-API-Key: YOUR_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "userId": "USER_ID",
    "amount": 99.99,
    "currency": "USD",
    "paymentMethod": "credit_card"
  }'
```

Both authentication methods ensure that only valid and authorized users or services can interact with the gateway.

RESULTS

This section presents the outcomes obtained after successfully implementing and executing the **Custom API Gateway for Microservices**.

The results include the testing of all API endpoints, database entries in MongoDB, monitoring dashboards from Grafana, and load testing metrics captured using K6 and InfluxDB.

Each component was verified for correctness, reliability, and performance under different testing conditions.

4.1 Overview of Testing Environment

All components — the **API Gateway**, **Payment Service**, and **Refund Service** — were executed in a local microservice setup.

The environment specifications were as follows:

- **Processor:** Intel i5 / i7 (or equivalent)
- **RAM:** 8 GB
- **Operating System:** Windows 10 / Ubuntu 22.04
- **Node.js Version:** 18.x or above
- **Database:** MongoDB 6.0 (local instance)
- **Monitoring Tools:** Grafana 10.x and InfluxDB 1.8 (via Docker)
- **Load Testing Tool:** K6

Each API was tested using **Postman** and **CURL commands**, and monitoring dashboards were analyzed via **Grafana**

4.2 API Testing Results

All major endpoints were tested successfully through the API Gateway. The outputs confirmed correct routing, authentication, and database interaction for both payment and refund operations.

4.2.1 Payment Service API Testing

(a) Create Payment

Endpoint:

```
curl -X POST http://localhost:4000/api/payments/pay \
  -H "X-API-Key: test-api-key-2a9ce0dc38f7338b237a07791ed1c83a" \
  -H "Content-Type: application/json" \
  -d '{
    "userId": " 690b3030fcb3c245721ee1c2 ",
    "amount": 1500,
    "currency": "Rupee",
    "paymentMethod": "credit_card"
  }'
```

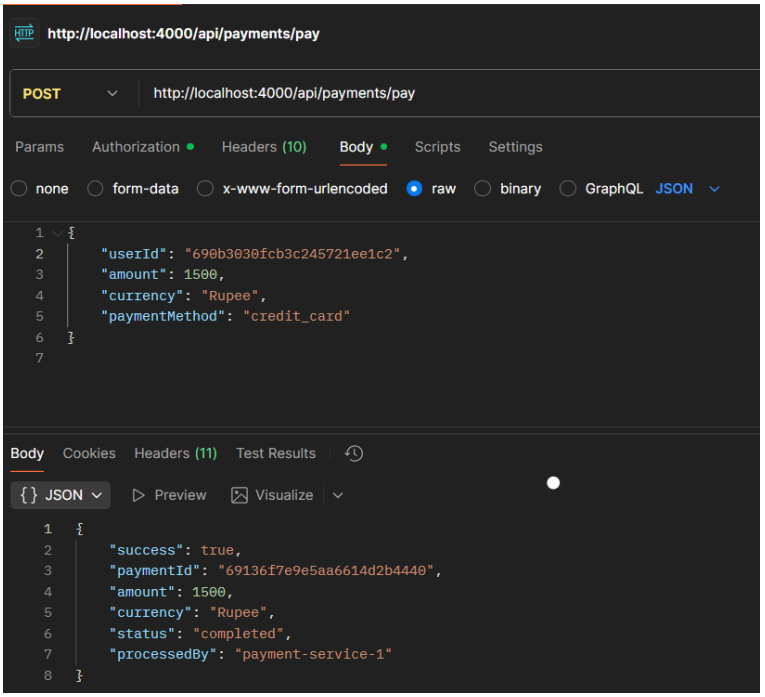


Figure 2 : Payment Success

(b) Check Payment Status

curl http://localhost:4000/api/payments/status/PAYMENT_ID \

-H "X-API-Key: test-api-key-2a9ce0dc38f7338b237a07791ed1c83a"

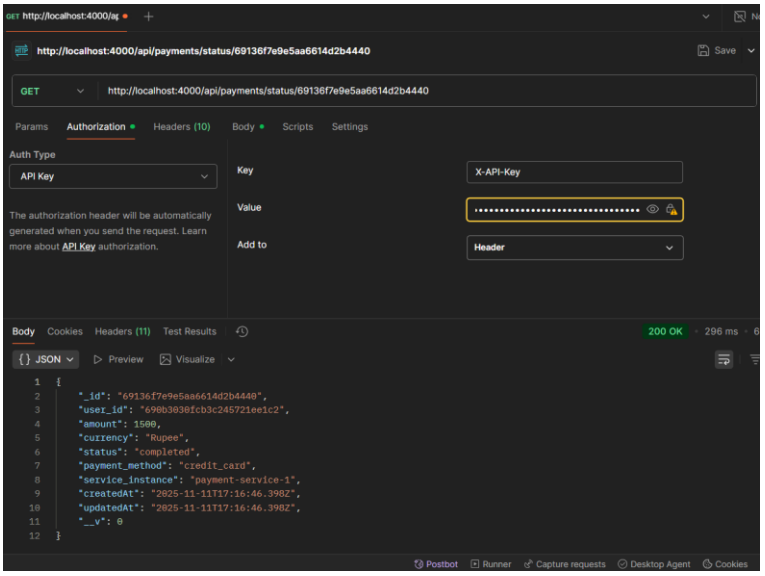


Figure 3 : Payment Status

4.2.2 Refund Service API Testing

(a) Process Refund

```
curl -X POST http://localhost:4000/api/refunds/refund \
  -H "X-API-Key: test-api-key-2a9ce0dc38f7338b237a07791ed1c83a " \
  -H "Content-Type: application/json" \
  -d '{
    "paymentId": "69136f7e9e5aa6614d2b4440 ",
    "amount": 150,
    "reason": "Customer request"
  }
```

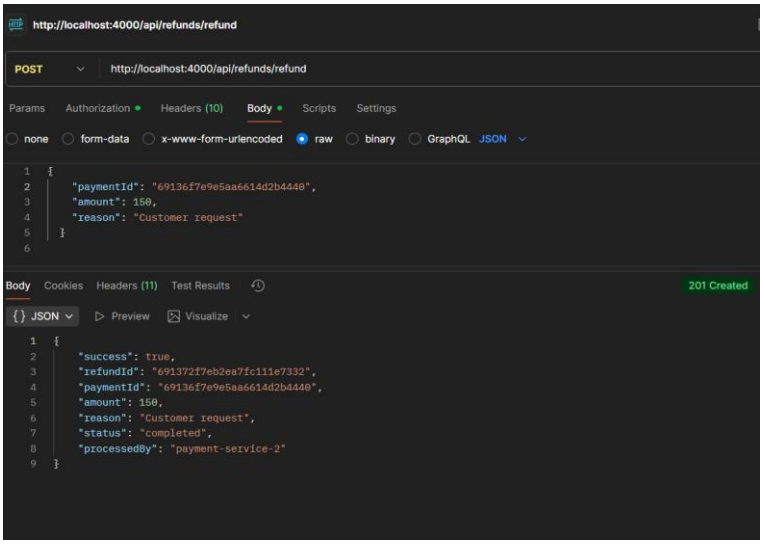


Figure 4 : Refund Success

(b) Get Transactions

curl http://localhost:4000/api/refunds/transactions?userId=690b3030fcb3c245721ee1c2&limit=10 \

-H "X-API-Key test-api-key-2a9ce0dc38f7338b237a07791ed1c83a "

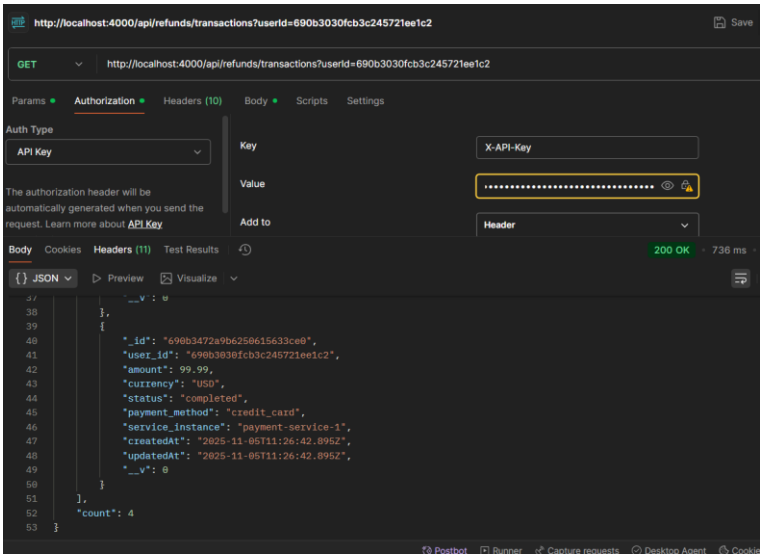


Figure 5: Get All Transactions

4.3 Database Verification

After successful API testing, all records were verified in the MongoDB database. Collections such as users, apikeys, payments, refunds, and requestlogs contained valid entries created during the tests.

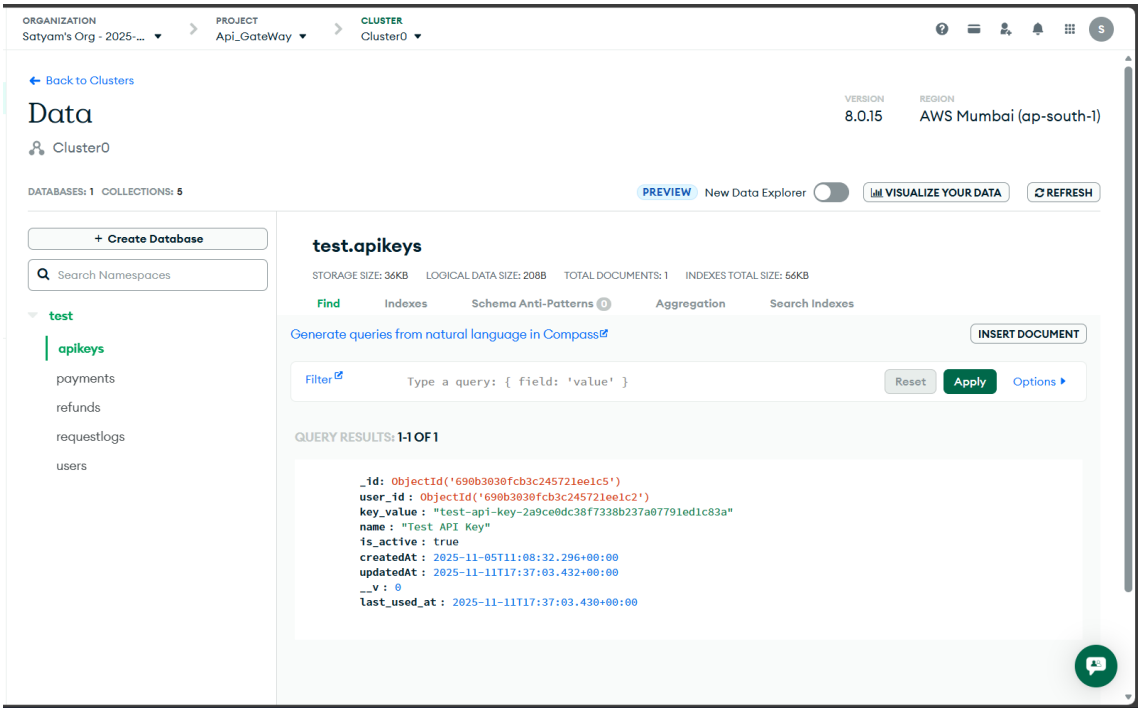


Figure 6 : MongoDB Atlas ApiKeys Storage

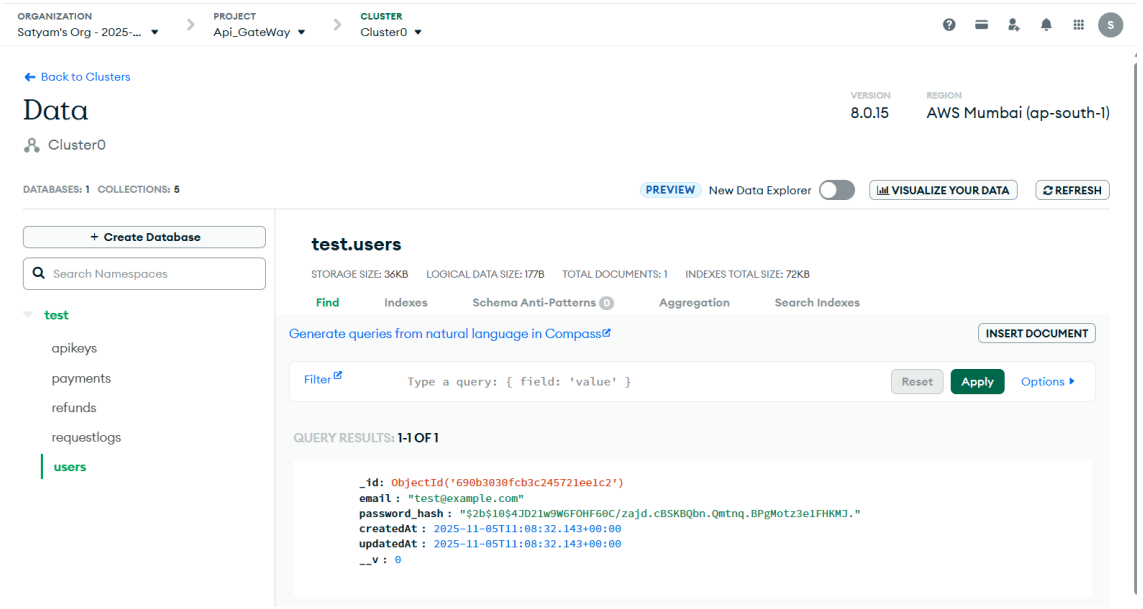
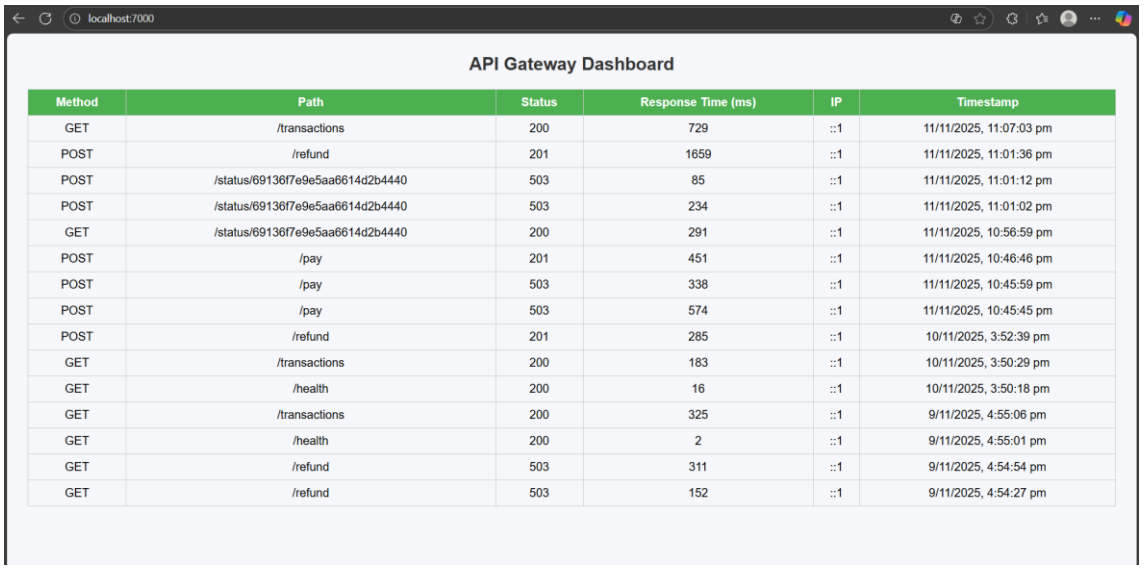


Figure 7 : MongoDB Atlas User Storage

4.4 Observability Dashboard Results

The **Grafana dashboard**, powered by **InfluxDB**, visualized key system metrics such as response time, request rate, and throughput collected during testing.

(a) API Gateway Dashboard (http://localhost:7000)



API Gateway Dashboard

Method	Path	Status	Response Time (ms)	IP	Timestamp
GET	/transactions	200	729	::1	11/11/2025, 11:07:03 pm
POST	/refund	201	1659	::1	11/11/2025, 11:01:36 pm
POST	/status/69136f7e9e5aa6614d2b4440	503	85	::1	11/11/2025, 11:01:12 pm
POST	/status/69136f7e9e5aa6614d2b4440	503	234	::1	11/11/2025, 11:01:02 pm
GET	/status/69136f7e9e5aa6614d2b4440	200	291	::1	11/11/2025, 10:56:59 pm
POST	/pay	201	451	::1	11/11/2025, 10:46:46 pm
POST	/pay	503	338	::1	11/11/2025, 10:45:59 pm
POST	/pay	503	574	::1	11/11/2025, 10:45:45 pm
POST	/refund	201	285	::1	10/11/2025, 3:52:39 pm
GET	/transactions	200	183	::1	10/11/2025, 3:50:29 pm
GET	/health	200	16	::1	10/11/2025, 3:50:18 pm
GET	/transactions	200	325	::1	9/11/2025, 4:55:06 pm
GET	/health	200	2	::1	9/11/2025, 4:55:01 pm
GET	/refund	503	311	::1	9/11/2025, 4:54:54 pm
GET	/refund	503	152	::1	9/11/2025, 4:54:27 pm

Figure 8 : API Gateway Dashboard

(a) Grafana dashboard (http://localhost:3000)

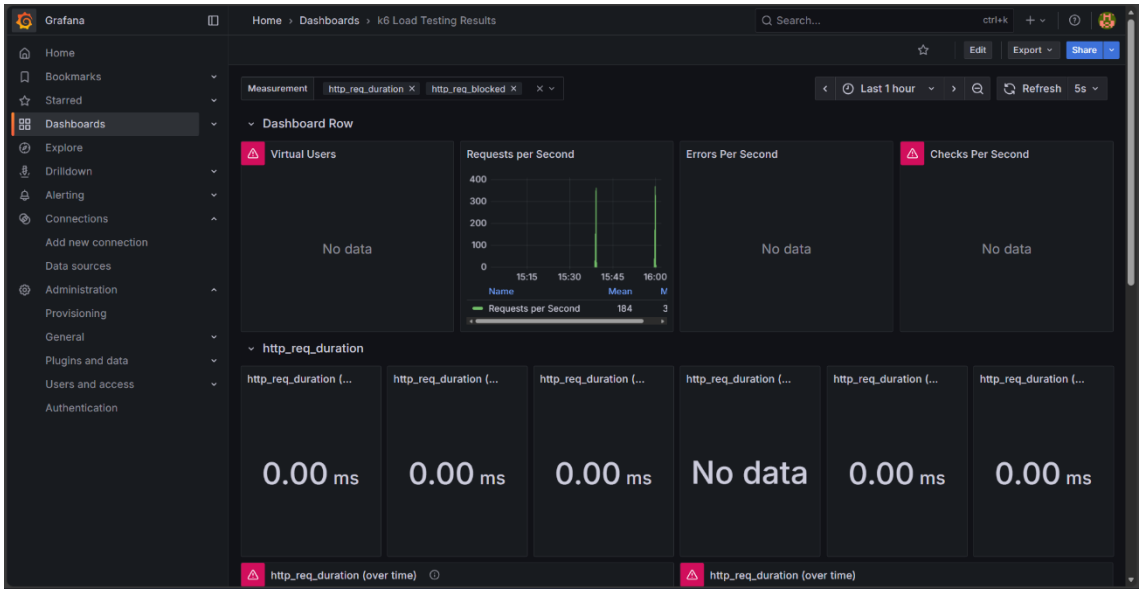


Figure 9: Grafana dashboard

4.5 Load and Stress Testing Results

Load testing was performed using K6, simulating concurrent users making payment and refund requests through the gateway.

(a) Api health check

Run - docker compose run k6 run --out influxdb=http://influxdb:8086/k6 /scripts/load_test.js

```
THRESHOLDS

http_req_failed
✗ 'rate<0.1' rate=100.00%


TOTAL RESULTS

checks_total.......: 9764    138.090873/s
checks_succeeded...: 0.00%   0 out of 9764
checks_failed......: 100.00% 9764 out of 9764

✗ status is 200
  ↳  0% — ✓ 0 / ✗ 9764

HTTP
http_req_duration....: avg=0s min=0s med=0s max=0s    p(90)=0s p(95)=0s
http_req_failed......: 100.00% 9764 out of 9764
http_reqs............: 9764    138.090873/s

EXECUTION
iteration_duration...: avg=1s min=1s med=1s max=1.01s p(90)=1s p(95)=1s
iterations...........: 9764    138.090873/s
vus..................: 12      min=5           max=299
vus_max..............: 300     min=300         max=300

NETWORK
data_received........: 0 B     0 B/s
data_sent............: 0 B     0 B/s
```

Satyam Gupta (15 hours ago) Ln 9, Col 28 (20 selected) Spaces: 2

Figure 10: Api health check

(b) Payment Load Test

Run - docker compose run k6 run --out influxdb=http://influxdb:8086/k6 /scripts/payment_load_test.js

```
THRESHOLDS

http_req_failed
✗ 'rate<0.1' rate=100.00%


TOTAL RESULTS

checks_total.......: 9764    138.090873/s
checks_succeeded...: 0.00%   0 out of 9764
checks_failed......: 100.00% 9764 out of 9764

✗ status is 200
  ↳  0% — ✓ 0 / ✗ 9764

HTTP
http_req_duration....: avg=0s min=0s med=0s max=0s    p(90)=0s p(95)=0s
http_req_failed......: 100.00% 9764 out of 9764
http_reqs............: 9764    138.090873/s

EXECUTION
iteration_duration...: avg=1s min=1s med=1s max=1.01s p(90)=1s p(95)=1s
iterations...........: 9764    138.090873/s
vus..................: 12      min=5           max=299
vus_max..............: 300     min=300         max=300

NETWORK
data_received........: 0 B     0 B/s
data_sent............: 0 B     0 B/s
```

Satyam Gupta (15 hours ago) Ln 9, Col 28 (20 selected) Spaces:

Figure 11: Payment Load

4.6 Rate Limiter Test

Too many Request by payment Load

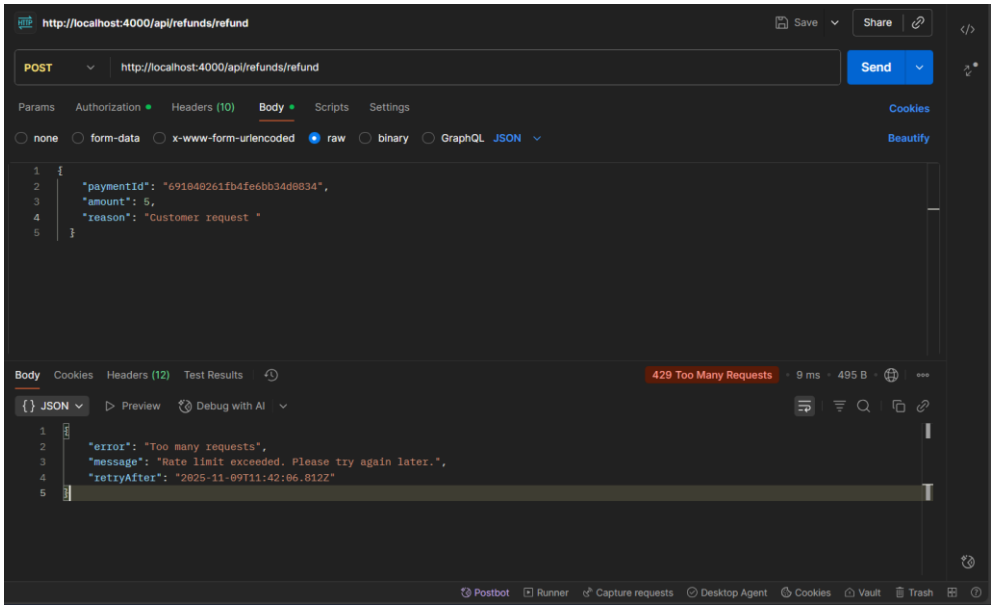


Figure 12: Rate limit exceeded

Conclusion

The project **"Custom API Gateway for Scalable Microservices with Authentication, Rate Limiting, and Observability"** was successfully designed and implemented to handle secure communication between multiple microservices.

The API Gateway effectively managed authentication using **JWT tokens** and **API keys**, applied **rate limiting**, and routed traffic through a **Round Robin Load Balancer**. Logging and monitoring were achieved using **Winston**, **Grafana**, and **InfluxDB**, ensuring complete visibility of system performance.

Two microservices — **Payment Service** and **Refund Service** — were developed and tested successfully, confirming proper database operations and smooth interaction through the gateway. The system performed efficiently even under load testing using **K6**, maintaining stable throughput and low latency.

Overall, the project met its objectives and provided practical understanding of **microservice architecture, API security, and performance monitoring** in real-world scenarios.

Future Enhancements

- Integration with **Kubernetes HPA** for auto-scaling.
- Add **OAuth2** and **gRPC** proxy support.
- Implement **caching layer** for repeated requests.
- Include **AI-based anomaly detection** in logs.

.

References

1. Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems.* O'Reilly Media.

2. Fowler, M. (2017). *Microservices Resource Guide.* martinfowler.com.

3. Node.js Foundation. (2023). *Node.js Documentation.* https://nodejs.org/en/docs

4. Express.js Team. (2023). *Express.js Guide.* https://expressjs.com

5. MongoDB Inc. (2023). *MongoDB Documentation.* https://www.mongodb.com/docs

6. Grafana Labs. (2023). *Grafana Documentation.* https://grafana.com/docs

7. InfluxData. (2023). *InfluxDB Documentation.* https://docs.influxdata.com

8. Grafana Labs. (2023). *K6 Load Testing Tool Documentation.* https://k6.io/docs